OPL Language Specification

Version 0.01

May 20, 2026

Contents

1 Introduction

OPL, the Observability Programming Language, is a domain specific language (DSL) designed to support the specification of telemetry pipelines. These pipelines [7] process *streams* of telemetry data by applying *operators* [8] to them. The set of operators is predefined by the language.

OPL supports user defined *functions* [5]. Functions are used in expressions that are parameters to operators. Thus, functions are first-order, whereas operators are potentially higher-order.

The data in a stream is always of some known *signal type* [11.4]. signal types belong to a type hierarchy derived from open telemetry (OTel) standards. The structure of each signal type includes a known, fixed part, and a dynamic part that allows for extensions. The static part consist of a set of named fields that remains fixed for the duration of an OPL program. ,The dynamic part contains a set of named attributes that may vary during program execution. The values of both fields and attributes belong to a family of *attribute types* [11.5.1]. Attribute types are a subset of *basic types*. Basic types are manipulated by functions and expressions. Basic types and signal types are distinct and disjoint.

OPL encourages static typechecking of pipelines; however, because signal types may include dynamically typed data, typechecking is not always possible, and so OPL provides an optional type system.

OPL programs may be invoked as nodes in a *global data flow graph* composed of varied processes and resources that produce or consume streams of data. An OPL program used in such a graph is called an *OPL processor* (when no confusion may arise, we may simply refer to such a program as a *processor*).

1.1 Levels

OPL is intended to evolve to support more extensive capabilities in the future. This document focuses on *level 1* capabilities, which deal with stateless stream processing. Level 2 is intended to support stateful stream processing. *This includes operations such as reduce/fold, notions such as windows etc.* Level 3 will support database features *Examples might include joins, persistence and/or materialized views.*

1.2 Notation

In this specification, normative text appears in roman font. Defined concepts are given in *italic font*. Non-normative commentary is given in *italic font*.

Commentary is useful for adding explanations, motivations, context etc.

Ongoing design commentary is given in purple italics

2 Program Structure

A program consists of optional declarations and policies [6] and one or more pipelines [7]. Declarations include functions [5] and let bindings [4]. The declarations may be preceded by a level declaration [1.1], indicating what level of this specification the program requires. If no level specification is given, it defaults to level 1.

⟨***OPLProgram***⟩ → ⟨*LevelDecl*⟩? (⟨*Decl*⟩
| ⟨*Policy*⟩)* ⟨*Pipeline*⟩+

⟨***LevelDecl***⟩ → `requires level` >= ⟨*NumberLiteral*⟩

⟨***Decl***⟩ → ⟨*FunctionDeclaration*⟩
| ⟨*LetBinding*⟩

We should we have a notion of language version that is orthogonal to levels. The language version may be given explicitly, but is optional, defaulting to the most recent version.

3 Scopes

A *scope* is a mapping from names to entities such as variables [4, function declarations [5] or pipelines [7]. Scopes exist at both compile-time and runtime. An OPL program always has a *global compile-time scope*.

Note that the term scope used herein should not be confused with the Otel concept of an instrumentation scope.

Declarations effect scopes by adding elements to them. Function declarations [5] introduce scopes at compile-time.

Execution introduces scopes at runtime. Scopes introduced at runtime are always based on scopes created at compile-time. They will typically differ in that they map names to values or functions rather than types. We say that a scope rs is *derived* from a scope cs if rs has entries for the same keys as cs.

Let s_1, s_2 be scopes. We write $s_1 \leftarrow s_2$, pronounced s_1 *overridden by* s_2, to denote the scope which maps all names defined in s_2 to their values in s_2 and any other name to its mapping in s_1, if any. We write $\{id_1 \mapsto v_1, \ldots, id_n \mapsto v_n\}$ to denote a scope that defines id_i and binds it to v_i, $\{id_1 \rightsquigarrow T_1, \ldots, id_n \rightsquigarrow T_n\}$

to denote a scope that associates id_i with a type T_i without regard to value, and $\{id_1 \mapsto v_1 : T_1, \ldots, id_n \mapsto v_n : T_n\}$ to denote a scope that defines id_i and binds it to v_i with type or type schema T_i, $i \in 1..n$.

Scopes may nest. Nested scopes override their immediately enclosing scope: if a scope s_{inner} is nested in a scope s_{outer}, then the effective scope is $s \triangleq s_{outer} \leftarrow s_{inner}$, and so all elements of s_{outer} are *available in* s unless an element with the same name is added to s_{inner}.

The compile-time scope of a function declaration is always nested in the global compile-time scope.

In short, we have lexical scoping.

The technical use of the term *scope* should not be confused with its informal use in phrases such as *beyond the scope of this specification.*

4 Let Bindings

A *let binding* binds a policy [6], pipeline [7], a port tied to an externally declared stream or a value of a basic type to a *variable* denoted by an identifier.

⟨***LetBinding***⟩ → `let` ⟨*Ident*⟩ ⟨*TypeSuffix*⟩? = (`receiver` ⟨*IDENT*⟩ ⟨*Block*⟩
| `exporter` ⟨*IDENT*⟩ ⟨*Block*⟩)

⟨***Block***⟩ → { ⟨*PropertyList*⟩ }

⟨***PropertyList***⟩ → (Property (, Property)*)?

⟨***Property***⟩ → ⟨*PropertyName*⟩ : ⟨*PropertyValue*⟩

⟨***PropertyName***⟩ → ⟨*IDENT*⟩
| ⟨*StringLiteral*⟩

⟨***PropertyValue***⟩ → ⟨*Scalar*⟩
| ⟨*Block*⟩

⟨***Scalar***⟩ → ⟨*NumberLiteral*⟩
| ⟨*StringLiteral*⟩
| `true`
| `false`
| `null`
| ⟨*ScalarArray*⟩

⟨***ScalarArray***⟩ → [⟨*Scalar*⟩ (, ⟨*Scalar*⟩)*]

A *port* may be either a *source port* or a *destination port*. A source port is indicated by the keyword `receiver`, and destination port is indicated by the keyword `exporter`. Ports are referenced by their declared name.

A port is analogous to a formal parameter of a function; in this analogy, the OPL program is the function. OPL programs may be bound to other processes or resources (which may themselves be OPL programs) in a manner analogous to function calls. How this binding is done is a matter for level 2 of this specification; for now it is out of scope.

The port is followed by a block that indicates the configuration of the port, given as a list of key-value pairs, known as *attributes*. This specification does not assign any specific meaning to any given attribute; the interpretation of the attributes in a port configuration is determined by the resource or process bound to the port. Ports are bound to resources outside the OPL program.

See discussion above.

Let $l \triangleq$ `let` ID `=` e

Processing of l has the following effect:

The variable ID is added to the compile-time global scope, with type [11] T, where T is the type of e.

Execution of l has the following effect:

If e is an expression, e is evaluated to v_e and v_e is associated with the variable ID in the global runtime scope.

The expression `let` ID`:` T `=` e is equivalent to `let` ID `=` $(e : T)$ [10.7].

It is a compile-time error if the compile-time global scope contains another declaration named ID.

5 Functions

A *function declaration* declares an OPL function. It consists of a function signature, followed by an equal sign and an expression denoting the function body.

⟨***FunctionDeclaration***⟩ → `func` ⟨*FunctionSignature*⟩ `=` ⟨*Expression*⟩

A *function signature* consists of an identifier F that specifies the name of the function, a parenthesized list of formal parameter declarations, and optionally, a return type T_r.

⟨***FunctionSignature***⟩ → ⟨*IDENT*⟩ ⟨*ParameterList*⟩ ⟨*TypeSigSuffix*⟩?

The parameter declarations each include the name of a parameter, p_i and may optionally declare a type T_i for the parameter, $i \in 1 \ldots k$.

⟨***ParameterList***⟩ → `(` ⟨*Parameter*⟩(`,` ⟨*Parameter*⟩)* `,`? `)`

⟨***Parameter***⟩ → ⟨*IDENT*⟩ ⟨*TypeSigSuffix*⟩?

If p_i does not declare a type, then $T_i = *$ [11.1]. If T_r is not explicitly declared, $T_r = T_e$, where T_e is the type of the function's body defined below.

Let $f \triangleq$ `func` F`(`p_1`:`T_1`, ... ,`p_n`:`T_n`):`T_r `=` e.

The signature of f corresponds to the function type [11.6] $T_1, \ldots, T_n \to T_r$.

Processing of f has the following effect:

A new scope, s_f, is introduced. The scope s_f is nested in the global scope of the enclosing program, s_g, yielding a scope $s \triangleq s_g \leftarrow s_f$. The scope s_f maps p_i to type $T_i, i \in 1 \ldots n$. The scope s is known as *the compile-time scope of the function*. The current compile-time scope is set to s, and the expression e is typechecked. Let T_e be the type of e in scope s.

Then the current compile-time scope is set back to s_g.

A binding of F to f is added to the global scope of the enclosing program with the type $T_1, \ldots, T_n \to T_r$ [11.6].

It is a type warning if $T_i = *$, $i \in 1 \ldots n$. It is a type warning if $T_r = *$. It is a compile-time error if the name F already exists in the global scope. It is a compile-time error if $p_i = p_j, i \neq j$.

6 Policies

A *policy* may control error handling [6.1], rate limiting [6.2] and timeouts [6.3]. Policies may be *global* or they may be restricted to a specific pipeline [7] or operator [8].

In the next level, we will also define policies for retrying.

⟨***Policy***⟩ → ⟨*ErrorPolicy*⟩
| ⟨*BatchPolicy*⟩
| ⟨*RetryPolicy*⟩
| ⟨*RateLimitPolicy*⟩
| ⟨*TimeoutPolicy*⟩

A policy declared at the top-level of an OPL program is said to be *global*. A policy p declared on a particular pipeline is *pipeline policy* for p. A policy declared on a single operator o is an *operator policy* for o.

A policy *governs execution* of certain operations. A global policy governs the execution of all operations in the program. A pipeline policy for pipeline p governs execution of all operations in p. An operator policy for operator o governs execution of the operation of o. If the execution of an operation is governed by a policy p, we say the operation is *governed* by p.

6.1 Error Handling Policies

An *error-handling policy* controls error handling

⟨***ErrorPolicy***⟩ → `policy error` ⟨*ErrorStrategy* ⟩

⟨***ErrorStrategy***⟩ → `strategy` ⟨*ErrorAction*⟩

⟨***ErrorAction***⟩ → `drop_signal`
| `route_to` ⟨*Destination*⟩
| `propagate`
| `log_and_continue`

Let $ep \triangleq$ `policy error strategy` es be an error policy.

If $es =$ `drop_signal` then any error signaled during execution of any operation governed by ep is ignored.

If $es =$ `route_to` d then any error signaled during execution of any operation governed by ep is directed to the port denoted by d.

If $es =$ `propagate` then any error signaled during execution of any operation governed by ep causes execution of the OPL program to be terminated. The signaled error is propagated to the process that invoked the OPL program.

The `propagate` action is analogous to raising an unhandled exception in the OPL program.

If $es =$ `log_and_continue` d then any error signaled during execution of any operation governed by ep is logged, and execution continues.

The binding of specific logs to an OPL program is outside the scope of this specification.

6.2 Rate Limiting Policies

A *rate-limiting policy* determines how the system may throttle operations to prevent overloading of subsequent ones.

⟨***RateLimitPolicy***⟩ → `policy rate_limit` ⟨*RateLimitConfig* ⟩

⟨***RateLimitConfig***⟩ → `{ strategy :` ⟨*RateLimitStrategy*⟩ `, on_limit :` ⟨*LimitAction*⟩
`}`

⟨***RateLimitStrategy***⟩ → `token_bucket` ⟨*TokenBucketConfig*⟩
| `fixed_window` ⟨*FixedWindowConfig*⟩
| `sliding_window` ⟨*SlidingWindowConfig*⟩

⟨***LimitAction***⟩ → `drop`
| `route_to` Destination
| `block`
| `back_pressure`

⟨***TokenBucketConfig***⟩ → `rate :` ⟨*Rate*⟩ `, burst :` Integer

⟨***FixedWindowConfig***⟩ → `rate :` ⟨*Rate*⟩ `, window :` ⟨*Duration*⟩

⟨***SlidingWindowConfig***⟩ → `rate :` ⟨*Rate*⟩ `, window :` ⟨*Duration*⟩ `, granularity :` ⟨*Duration*⟩

⟨***Rate***⟩ → Integer "/" ⟨*TimeUnit*⟩

Let $rp \triangleq$ `policy rate_limit { strategy :` rs `, on_limit :` la `}` be a rate-limiting policy.

6.3 Timeout Policies

A *timeout policy* determines if and when operations will time out if they do not meet latency requirements, and what to do if such a timeout occurs.

⟨***TimeoutPolicy***⟩ → `policy timeout` ⟨*TimeoutConfig* ⟩

⟨***TimeoutConfig***⟩ → `{ operation :` ⟨*Duration*⟩ `, pipeline :` ⟨*Duration*⟩ `, on_timeout :` ⟨*TimeoutAction*⟩ `}`

⟨***TimeoutAction***⟩ → `fail`
| `route_to` ⟨*Destination*⟩

Let $bp \triangleq$ `policy timeout { operation :`d_1 `, pipeline :` d_2 `, on_timeout :` ta `}` be a timeout policy.

If $ta =$ `fail` then if any operation governed by tp takes longer than d_1 an *OperationTimeout* runtime error occurs. The error should provide the name and of the operation that failed, its source code location and the timeout period that was exceeded.

If any pipeline governed by tp takes longer than d_2 a *PipelineTimeout* runtime error occurs. The error should provide the pipeline's source code location and the timeout period that was exceeded.

if $ta =$ `route_to` d then:

- if any operation governed by tp takes longer than d_1 the results of that operation are redirected to the timeout sink.
- If any pipeline governed by tp takes longer than d_2 the results of that pipeline are redirected to the timeout sink.

It is a compile-time error if either d_1 or d_2 are not valid `duration` tagged literals [10.6.4].

Do we insist on having literals here? Without mandatory typechecking, we could not ensure that variables are of type `duration`.

7 Pipelines

A *pipeline* is a function on streams.

These functions are distinct from user defined functions [5].

In all but the most trivial cases, a pipeline arises from the composition of a number of stream functions.

⟨***Pipeline***⟩ → ⟨*Source*⟩ (| ⟨*Stage*⟩)* ⟨*Terminal*⟩?

⟨***Source***⟩ → ⟨*Ident*⟩

⟨***Stage***⟩ → ⟨*OperatorCall*⟩
| ⟨*Control*⟩
| ⟨*PolicyStage*⟩
| ⟨*StreamComb*⟩

A pipeline consists of a *source* and a series of *stages* [9]. A source represents a stream of data. The first stage processes the source. Each subsequent stage processes the result of the previous stage.

A pipeline $s_1|\ldots|\ s_n$ evaluates to $(s_n \circ s_{n-1} \ldots \circ s_2)(s_1)$ for $n > 1$, and to s_1 otherwise.

The pipe syntax $s_1|s_2$ is really stream function composition in reverse order, or if you prefer, an alternate form of stream function application which is left-associative.

8 Operators

Operators are built in functions that operate on signal types [11.4].

As such, operators are distinct from OPL's user-defined functions [5] which are restricted to basic types [11.5].

The set of operators in OPL is fixed. Operators are often parameterized by expressions. Operators are applied to streams of signal type via operator calls [9.2]. OPL defines the following operators:

8.1 Extend

The `extend` operator allows the assignment of an element of a map.

The purpose of `extend` is to add or mutate dynamic attributes of a stream eleement.

Or, equivalently, to an element of a list of key-value pairs.

Let x be the current element of the input stream s. The operator call `extend` $id[k] = e$ produces an element x' such that x' is equivalent to x except for the value of the field id, whose value in x' is the same as $x[$, except that $x'.id[k] = v$ where v is the result of evaluating e.

8.2 Identity

The `identity` operator returns its input unchanged.

This may seem pointless at first glance, but in fact is a basic operation needed in order to define the behavior of other key operations.

8.3 Keep

The `keep` operator allows for removing all but a specified set of dynamic attributes of a stream element.

`keep` removes all attributes except those explicitly listed, which are kept.

`keep` can also be referenced using the predefined alias `project_keep`.

8.4 Remove

The `remove` operator allows for removing a set of dynamic attributes of a stream eleement.

`remove` can also be referenced using the predefined alias `project_remove`.

8.5 Rename

The `rename` operator allows for renaming a set of dynamic attributes of a stream eleement.

`rename` can also be referenced using the predefined alias `project_rename`.

8.6 Set

The `set` operator allows the assignment of a field of a signal.

Let x be the current element of the input stream s. The operator call `set` $id = e$ produces an element x' such that x' is equivalent to x except for the value of the field id, whose value in x' is v, where v is the result of evaluating e.

Let $Stream < \texttt{T} >$ be the type of s. It is a runtime error if id is not a member of T.

Even though we give a type warning at compile time, in an optional type system we do not preclude the code from being run (say, for test purposes). If it runs, it must fail at this point.

It is a type warning if id is not a member of T. It is a type warning if the type of e is not compatible with T.

8.7 Casts: Accept and Reject

The `accept` and `reject` operators are identity operators that cast the input type to the type given as their argument.

The operator `accept` T is equivalent to `where is` T.

The operator `reject` T is equivalent to `where not is` T.

It is a compile time error if T is not a signal type [11.4].

9 Stages

Stages operate on streams. A stage has an *input stream* and, unless it is the terminal stage of a pipeline, and an *output stream*. OPL provides distinct syntactic forms for different kinds of stream processing functions.

Evaluation of a stage always takes place in a scope. $s_g \leftarrow s_{stage}$, where s_{stage} maps the name of every attribute of the current stream element to its value.

9.1 Control Flow

OPL supports control flow by means of several stream combinators, that allow streams to be forked and reunified, possibly conditionally.

Stream combinators are distinct from operators, because they do not work on individual stream elements, but rather on entire streams.

⟨***Control***⟩ → ⟨*IfExpression*⟩
| ⟨*Union*⟩
| ⟨*ForkExpression*⟩

9.1.1 If

The `if` combinator allows a stream to split into multiple streams, each subject to distinct treatment by a different pipeline based on specific conditions.

⟨***IfExpression***⟩ → `if` ⟨*Expression*⟩ `(` ⟨*Pipeline*⟩ `)` (`else if (` ⟨*Pipeline*⟩ `)`) *
(`else (` ⟨*Pipeline*⟩ `)`)?

Given an `if` expression of the form `if` c_1 `(`p_1`)` `elseif` c_2 `(`p_2`)` ... `elseif` c_{n-1} `(`p_{n-1}`)` `else` c_n `(`p_n`)`, the conditions $c_i, i \in 1..n$ are evaluated on each element of the input stream(s). If an element matches c_i and does not match any condition $c_j, j < i$, the element is appended to the input stream for p_i, $i \in 1..n$. Any element of s that does not match any $c_i, i \in 1..n$ becomes part of a residual stream s_{n+1}. Let s_i be the input stream of $p_i, i \in 1..n+1$. For any two elements of e_k, e_l of s_i, if e_k precedes e_l in s, e_k precedes e_l ins s_i.

The result of the `if` combinator is the `union` [9.1.2] of $s_i, i \in 1..n+1$.

9.1.2 Union

The `union` combinator takes two streams and produces a stream composed of the union of their elements.

⟨***Union***⟩ → `union` ⟨*Pipeline*⟩*

The only constraint on the order of the elements in the result stream is that that if e_i precedes $e_j \in s$, then e_i precedes e_j in the result, where s is one of the input streams of the `union` combinator.

There is no other constraint on the order of the result. Thus the ordering between elements of the streams s_1 and s_2 is undefined.

9.1.3 Fork

The `fork` combinator feeds the elements of its input stream into two or more *The original syntax said it limited to two, but other parts of the doc contradict that, and it doesn't seem to be necessary, or a good fit with the rest of the system. Is there a real reason to limit?* child pipelines.

⟨***ForkExpression***⟩ → `fork (`⟨*Pipeline*⟩ `)` *

Given an `fork` expression of the form `fork(`p_1`)` ... `(`p_n`)`, its input stream s is replicated into n identical streams s_i and s_i becomes the input for $p_i, i \in 1..n$.

The result of the `fork` combinator is the `union` [9.1.2] on the results of the children $p_i, i \in 1..n$.

It follows from the definition `union` that no de-duplication occurs even if the children are identical, and that the ordering among the children is undefined, but elements of each child pipeline preserve their relative order

9.2 Operator Calls

An operator call takes a named operator and lifts it to the stream domain.

⟨***OperatorCall***⟩ → ⟨*Ident*⟩ ⟨*Args*⟩?

The first argument of the operator is implicitly an element of the stage's incoming stream, and other arguments are provided explicitly.

The entire call expression is treated as a function from the stream signal type to itself, and it is implicitly mapped over the stream.

An operator call of the form *id* is equivalent to the operator call *id*().

Let $oc \triangleq id(a_1, \ldots a_n)$ be an operator call. Let o be the operator denoted by *id* and let $f = \lambda x.o(x, a_1, \ldots a_n)$. Let s be the input stream of the call.

Recall that operator calls are a form of stage, and thus always have an input stream.

Then the value of *oc* is $map(f, s)$. The value of *oc* is the output stream of the operator call/stage.

9.3 Routing

Routing operations direct stream data to destinations. There are two such operations.

`route_to` directs data to a specified destination, which may be a pipeline or an external stream.

We can think of `route_to` by itself as a meta-operation, which, given a destination, produces an operation that takes its input and writes to the destination. The syntax ensures that only applications of `route_to` (i.e., operations) appear in pipelines.

`drop` simply absorbs its argument.

⟨***Terminal***⟩ → `route_to`⟨*Ident*⟩
| `drop`

It is a compile-time error if a routing operation appears anywhere except the last position in a pipeline, unless the pipeline is nested in a fork [9.1.3] or conditional [9.1.1]. It is a compile-time error if the destination of a routing operation does not denote an externally specified destination stream, unless the pipeline is nested in a fork or conditional.

9.4 Where

The `where` combinator filters a stream based on a predicate.

`where` *c* is equivalent to `if` *c* `identity else drop`.

`where` can also be referenced using the predefined alias `filter`.

10 Expressions

Expressions describe computations on basic types [11.5].

10.1 Boolean Literals

The identifiers `true` and `false` are predefined, denoting the booleans true and false respectively, and have type `bool` [11.5.2].

BoolLiteral → `true` | `false`

10.2 Numeric Literals

A *numeric literal* is either an *integer literal* or a *floating-point literal*.

An integer literal represents an integer that can be represented by one of the integer types. An integer literal has the smallest signed integer type that can represent it.

A floating point literal represents an IEEE floating point number and has type `float64`.

The numeric types form a lattice, defined by the relationships `uint32` ≤ `uint64`, `int32` ≤ `int64`, `int32` ≤ `float64`,

The type `float64` is a *floating-point type*. Other numeric types are *integer types*. The types `uint32` and `uint64` are *unsigned types*; all other numeric types are *signed types*.

Numeric literals can be expressed in decimal and hexadecimal systems. The prefix ‘0x‘ or ‘0X‘ indicates a hexadecimal number literal. Numeric decimals may also be expressed in scientific notation using ‘E‘ or ‘e‘.

⟨***NumberLiteral***⟩ → ⟨*FLOAT*⟩
| ⟨*INT*⟩

⟨***DECIMAL***⟩ ::= `0` ... `9`

⟨***HEX***⟩ ::= `0` ... `9`
| `a` ... `f`
| `A` ... `F`

⟨***FRAC***⟩ ::= ⟨*DECIMAL*⟩+

⟨***EXPO***⟩ ::= (`e` | `E`) (`+` | `-`)? ⟨*FRAC*⟩

⟨***FLOAT***⟩ ::= ⟨*FRAC*⟩ `.` ⟨*FRAC*⟩? ⟨*EXPO*⟩?
| ⟨*FRAC*⟩ ⟨*EXPO*⟩?
| `.` ⟨*FRAC*⟩ ⟨*EXPO*⟩?

⟨***INT***⟩ ::= (`1` ... `9`)⟨*FRAC*⟩?
| `0` lookahead ∉ `0` ... `9`
| `0` (`x` | `X`) ⟨*HEX*⟩+

The predefined identifier `system::nan` denotes an IEEE floating point NaN value and has type `float64`. The predefined identifier `system::infinity` denotes IEEE floating point positive infinity and has type `float64`.

OPL does not distinguish among NaN values. An implication is that any foreign function called from OPL should not expect any specific NaN variants.

10.3 String Literals

String literals are represented in the UTF-8 encoding. String literals are delimited by either a pair of double quotes or a pair of single quotes. A backslash (\) character is used for conventional character escaping.

⟨***StringLiteral***⟩ → ⟨*DQSTRING*⟩
| ⟨*SQSTRING*⟩

⟨***DQSTRING***⟩ ::= `"` ⟨*DQCHAR*⟩* `"`

⟨***DQCHAR***⟩ ::= any one character except `"`, ⟨*LINETERMINAL*⟩ or \
| \ ⟨*ESCAPE*⟩

⟨***LINETERMINAL***⟩ ::= the ASCII LF character, also known as "newline"
| the ASCII CR character, also known as "return"
| the ASCII CR character followed by the ASCII LF character

⟨***ESCAPE***⟩ ::= the ASCII SP character, also known as "space"
| one of `t`, `v`, `r`, `n`, `f`, `b`, \, `'`, `"`

⟨***SQSTRING***⟩ ::= ' ⟨*SQCHAR*⟩* '

⟨***SQCHAR***⟩ ::= any one character except ', ⟨*LINETERMINAL*⟩ or \
| \ ⟨*ESCAPE*⟩

String literals are enclosed in double quotes ("), for example "A simple string". A backslash (\) character is used for conventional character escaping.

We want to support string interpolation.

10.4 `null`

The value `null` denotes the absence of a normal value. OPL always pairs `null` with a *provenance* [11.3] to produce a value of type `Null`.

10.5 Identifier Expressions

An *identifier expression* consists of a single identifier.

⟨***IdentifierExpression***⟩ → ⟨*IDENT*⟩

Let $i \triangleq ID$. Evaluation of i proceeds as follows:

Let s be the current runtime scope. The value v of ID is looked up in s. The value of i is v.

The type of i is the type associated with ID in the enclosing compile-time scope.

It is a compile-time error if ID is not bound to a variable declaration.

Note that this implies that it is an error if ID is undeclared, but also if it denotes a function or type. Neither functions nor types are values in OPL.

10.6 Tagged Literals

Tagged literals are either tagged string literals [10.6.1] or tagged numeric literals [10.6.2]. String tagged literals consist of a string literal prefixed by an identifier, known as the *tag*. Numeric tagged literals consist of a numeric literal suffixed by a tag.

⟨***TaggedLiteral***⟩ → ⟨*IDENT*⟩ ⟨*StringLiteral*⟩
| ⟨*NumberLiteral*⟩ ⟨*IDENT*⟩

Each tag must name a unique *tag handler* function.

Tagged literals represent either regexes [10.6.3] or date/time values [10.6.4].

10.6.1 Tagged String Literals

Let $tl \triangleq Tl$. It is a compile-time error if T is not one of the predefined functions `regex`, `date`, `time`, `timestamp`, `duration` or `interval`.

This restriction can and should be lifted, to allow for arbitrary tag functions. However, the special status of r and the various date/time tags remains, as they are used to facilitate compile time checking of the literal string format.

10.6.2 Tagged Numeric Literals

Let $tl \triangleq lT$. It is a compile-time error if T is not one of the predefined functions `ms`, `s`, `m` or `h`.

As above, this restriction should be lifted. The notation would then allow for a wide range of user-defined units.

In either case:

Evaluation of tl is proceeds as follows: Let vl be the result of evaluating l, and let h be the tag handler for T. The value of lT is $h(vl)$.

If the same literal repeats, it is not re-evaluated, as tagged literals are compile-time constants. In principle it should not matter (but if the constructor is stateful, it might). Thus OPL calls the tag handler once per literal, at compile-time.

The type of a tagged literal Ts, where T the tag and s is the literal, is the type returned by the handler associated with T.

To be added when we allow user-defined tags: It is a compile-time error if t does not denote the name of a function in the compile-time scope of tl.

10.6.3 Regexes

A regular expression literals are tagged literals whose tag is the identifier `regex`. The meaning of the regular expression literal is defined by interpreting the contents of its string literal component as a regular expression as defined by IETF RFC 9485 (https://datatracker.ietf.org/doc/html/rfc9485).

If we allow string interpolation, string literals are no longer compile-time constants, and our ability to validate the contents is (partially) lost. One solution is to distinguish constant string literals from non-constant ones, and enforce syntax checks only on constants, perhaps warning in the non-constant case. We could try to be even smarter, depending on what is being interpolated, but that seems a rathole.

It is a compile-time error if the contents of the string literal component of a regular expression literal does not conform to IETF RFC 9485 (https://datatracker.ietf.org/doc/html/rfc9485).

Regular expression literals have type `regex` [11.5.6].

10.6.4 Date/Time Literals

Date/Time literals are either string tagged literals whose tag is one of the identifiers: `date`, `time`, `timestamp`, `duration` or `interval`, or numeric tagged literals whose tag is one of `ms`, `s`, `m` or `h`. The meaning of the date/time literal is defined as follows :

- If the literal's tag is `date`, then its string literal component is interpreted as a date, as defined by ISO 8601.
- If the literal's tag is `time`, then its string literal component is interpreted as a time, as defined by ISO 8601.
- If the literal's tag is `timestamp`, then its string literal component is interpreted as a combined ate and time, as defined by ISO 8601.

- If the literal's tag is `duration`, then its string literal component is interpreted as a duration, as defined by ISO 8601.
- If the literal's tag is `interval`, then its string literal component is interpreted as an interval, as defined by ISO 8601.
- If the literal's tag is `ms` then d denotes a `duration` of k milliseconds.
- If the literal's tag is `s` then d denotes a `duration` of k seconds.
- If the literal's tag is `m` then d denotes a `duration` of k minutes.
- Ifthe literal's tag is `h` then d denotes a `duration` of k hours.

It is a compile-time error if the contents of string literal component of a date/time literal with tag `date` does not conform to the format of a date in ISO 8601. It is a compile-time error if the contents of string literal component of a date/time literal with tag `time` does not conform to the format of a date in ISO 8601. It is a compile-time error if the contents of string literal component of a date/time literal with tag `timestamp` does not conform to the format of a combined date and time in ISO 8601. It is a compile-time error if the contents of string literal component of a date/time literal with tag `duration` does not conform to the format of a duration in ISO 8601. It is a compile-time error if the contents of string literal component of a date/time literal with tag `interval` does not conform to the format of an interval in ISO 8601.

Are there size limits of ISO 8601 that we should enforce for numeric date/time literals?

The type of a date/time literal is the type specified by its tag [11.5.7].

ISO 8601 does not support Date/Time arithmetic and comparisons. We expect to add support for that shortly.

We expect to generalize this approach to additional units, per the Unified Code for Units of Measure (UCUM) specification.

10.7 Parenthesized Expressions

⟨***ParenthesizedExpression***⟩ → `(` ⟨*Expression*⟩
| ⟨*ExpressionTypeSigSuffix*⟩ `)`

The expression `(`e`)` is equivalent to the expression `(`e`:`T`)` where T is the type of e.

Let $p \triangleq$ `(`e`:`T`)`.

Evaluation of p proceeds as follows:

The expression e is evaluated to v. The value of p is v.

The type of p is T.

It is a type warning if the type of e is not a subtype of type T.

10.8 Index Expressions

An *index expression* provides random access to the members of a map [11.5.1].

⟨***IndexExpression***⟩ → ⟨*MemberExpression*⟩ `[` ⟨*Expression*⟩ `]`

Let $i \triangleq e_1\texttt{[}e_2\texttt{]}$.

Evaluation of i proceeds as follows:

The expression e_1 is evaluated to a value v_1. If v_1 is `null`, the value of i is `null` with provenance e_1. Otherwise, the expression e_2 is evaluated to a value v_2 if v_2 is `null`, the value of i is `null` with provenance e_2. Otherwise, the value of i is the element of e_1 stored under key v_2, if there is one, or `null` with provenance i otherwise.

Let T_1 be the type of e_1. It is a type warning if T_1 is not a map type. Let T_2 be the type of e_2. It is a type warning if T_2 is not compatible with type `string`.

In particular, if either T_1 or T_2 or both are nullable, a type warning arises.

Unclear what checking we can do here. At a minimum, T_1 must be attributeValue and the overall type of the expresision is attributeValue. Hopefully, we can do better.

10.9 Attribute Selection Expressions

An *attribute selection expression* provides a way to access a member of a stream element [11.4].

⟨***AttributeSelectionExpression***⟩ → ⟨*MemberExpression*⟩ `.` ⟨*PropertyName*⟩

Let $m \triangleq e\texttt{.}p$.

Evaluation of m proceeds as follows:

The expression e is evaluated to v. If v is `null`, then the m evaluates to `null` with provenance m. Otherwise, the value of m is the value v_p of the field p of v.

Let T be the type of e. If $T = \{p_1\texttt{:}T_1\texttt{,}\ \ldots\ \texttt{,}p_n\texttt{:}T_n\}$ where $p_k = p$, then the type of m is T_k. If $T = \{p_1\texttt{:}T_1\texttt{,}\ \ldots\ \texttt{,}p_n\texttt{:}T_n\}\texttt{?}$ where $p_k = p$, then the type of m is $T_k\texttt{?}$.

Otherwise, the type of m is *.

It is a type warning if T not a non-nullable type with a member named p.

10.10 Member Expressions

A *member expression* is either an identifier expression [10.5], a parenthesized expression [10.7], a function call [10.20], a literal ([10.1, 10.2, 10.3]), an attribute selection expression [10.9] or an index expression [10.8]. It serves exclusively as a syntactic grouping.

⟨***MemberExpression***⟩ → ⟨*IndexExpression*⟩
| ⟨*AttributeSelectionExpression*⟩
| ⟨*FunctionCall*⟩

| ⟨*ExtensionExpression*⟩
| ⟨*PrimitiveExpression*⟩

⟨***PrimitiveExpression***⟩ → ⟨*IdentifierExpression*⟩
| ⟨*StringLiteral*⟩
| ⟨*TaggedLiteral*⟩
| `true`
| `false`
| `null`
| `(` ⟨*ParenthesizedExpression*⟩

10.11 Unary Expressions

A *unary expression* is either a member expression [10.10] or the application of a unary operator to two unary expressions. A *unary operator* is one of the operators `-`, `~` or `not`. The `-` operator implements negation. If T is a numeric type, the `-` operator has type $T \to T$. Otherwise the type of `-` is $T \to *$.

The `~` operator implements bitwise negation. If e has type T, where T is either a numeric type, `bool` or `bool ?`, `~` has type $T \to T$. Otherwise the type of `~` is $T \to *$.

The `not` operator implements boolean negation. `not` has type `bool` → `bool`.

⟨***UnaryExpression***⟩ → ⟨*NumberLiteral*⟩
| `-` ⟨*UnaryExpression*⟩
| `~` ⟨*UnaryExpression*⟩
| `not` ⟨*UnaryExpression*⟩
| ⟨*MemberExpression*⟩

Let $u \triangleq \theta e$ where θ is a unary operator.

Evaluation of u proceeds as follows:

The expression e is evaluated to v. If v is `null`, the value of u is `null` with provenance e. Otherwise, the value of u is the result of $\theta(v)$.

Let T be the type of e and let R be the type of θe. The type of u is R if T is non-nullable, and R`?` otherwise. It is a type warning if the call $\theta(u)$ would give rise to a type warning. It is a type warning if $\theta =$ `-` and T is not a numeric type. It is a type warning if $\theta =$ `~` and T is neither `bool` nor a numeric type.

In particular, if T is nullable, a type warning arises, even if T! would be an appropriate argument for θ.

10.12 Multiplicative Expressions

A *multiplicative expression* m is either a unary expression [10.11] or the application of a multiplicative operator to two multiplicative expressions. A *multiplicative operator* is one of the operators `*`, `/`, `%`. The `*`, `/`, and `%` operators implement multiplication, division and remainder, respectively.

Specifically, each such operator converts its arguments to their least common supertype and performs the operation.

⟨***MultiplicativeExpression***⟩ → ⟨*UnaryExpression*⟩
| ⟨*MultiplicativeExpression*⟩ ⟨*MultiplicativeOp*⟩ ⟨*UnaryExpression*⟩

⟨***MultiplicativeOp***⟩ → `*`
| `/`
| `%`

Let $m \triangleq e_1 \theta e_2$ where θ is a multiplicative operator.

Evaluation of m proceeds as follows:

The expression e_1 is evaluated to v_1. If v_1 is `null`, the value of m is `null` with provenance e_1. Otherwise, e_2 is evaluated to v_2. If v_2 is `null`, the value of m is `null` with provenance e_2. Otherwise, the value of m is the result of $\theta(v_1, v_2)$.

Let e_1 have type T_1 and e_2 have type T_2. If T_1 and T_2 are numeric types: m has type $LCS(T_1, T_2)$. Otherwise, the type of m is *.

It is a type warning if either T_1 or T_2 are not non-nullable numeric types.

10.13 Additive Expressions

An *additive expression* is either a multiplicative expression [10.12] or the application of an additive operator to two additive expressions. An *additive operator* is one of the operators `+` or `-` which implement addition and subtraction, respectively.

⟨***AdditiveExpression***⟩ → ⟨*MultiplicativeExpression*⟩
| ⟨*AdditiveExpression*⟩ ⟨*AdditiveOp*⟩ ⟨*MultiplicativeExpression*⟩

⟨***AdditiveOp***⟩ → `+`
| `-`

Let $a \triangleq e_1 \theta e_2$ where θ is an additive operator.

Evaluation of a proceeds as follows:

The expression e_1 is evaluated to v_1. If v_1 is `null`, the value of a is `null` with provenance e_1. Otherwise, e_2 is evaluated to v_2. If v_2 is `null`, the value of a is `null` with provenance e_2. Otherwise, the value of a is the result of $\theta(v_1, v_2)$.

Let e_1 have type T_1 and e_2 have type T_2. If T_1 and T_2 are numeric types: a has type $LCS(T_1, T_2)$. Otherwise, the type of a is *. It is a type warning if the types of e_1 and e_2 are not non-nullable numeric types.

This will need to change if we allow adding dates and times.

10.14 Relational Expressions

A *relational expression* is either an additive expression [10.13] or the application of a relational operator to two relational expressions. A *relational operator* is one of the operators `==`, `!=`, `<`, `>`, `<=`, `>=`.

The operators `==`, `!=`, `<`, `>`, `<=`, `>=` implement the equal, not equal, less than, greater than, less than or equal, greater or equal operators respectively. The `==` and `<` operators are defined by equality and the linear orderings among values of types as given in sections 11.5.2, 11.5.5, 11.5.3, 11.5.4.

This will need to change if we allow comparing dates and times.

Let $v_1 \neq$ `system` :: `nan`, $v_2 \neq$ `system` :: `nan` be values.

v_1`!=`$v2$ iff it is not the case that $v_1 = v_2$.

v_1`<`v_2 iff $v_1 < v_2$.

v_1`<=`v_2 iff either $v_1 = v_2$ or $v_1 < v_2$.

v_1`>`v_2 iff $v_1 > v_2$.

v_1`>=`v_2 iff either $v_1 = v_2$ or $v_1 > v_2$.

In any other case (i.e, if one or both of $v_1, v2$ are `system::nan`) then all the relational operators return false, per the rules of IEEE floating point.

⟨***RelExpression***⟩ → ⟨*AdditiveExpression*⟩
| ⟨*RelExpression*⟩ ⟨*RelOp*⟩ ⟨*AdditiveExpression*⟩

⟨***RelOp***⟩ → `==`
| `!=`
| `<`
| `>`
| `<=`
| `>=`

Let $r \triangleq e_1 \theta e_2$ where θ is a relational operator.

Let T_i be the type of $e_i, i \in 1..2$.

Evaluation of r proceeds as follows:

If e_1 is `null`, the value of r is `null` with provenance e_1. Otherwise, if e_2 is `null`, the value of r is `null` with provenance e_2. Otherwise r is equivalent to θ`(`e_1`,`e_2`)`. For $\theta \in$ `==`, `!=`, the signature of θ is θ`(`p_1`:`any`,`p_2`:`any`):bool`, if the T_1 and T_2 are both non-nullable [11.2] and θ`(`p_1`:`any`,`p_2`:`any`):bool?` otherwise.

For $\theta \in$ `<`, `>`, `<=`, `>=` the signature of θ is θ`(`p_1`:`T`,`p_2`:`T`):bool` if the types of T_1 and T_2 are both non-nullable [11.2] and θ`(`p_1`:`T`,`p_2`:`T`):bool?` otherwise, where $T = LCS(T_1, T_2)$. It is a type warning if either T_1 or T_2 is nullable.

10.15 Bitwise And

A *bitwise and expression* is either a relational expression [10.14] or the application of the bitwise and operator, `&`, to two bitwise and expressions.

Let $a \triangleq e_1$ `&` e_2.

Evaluation of a proceeds as follows:

The expression e_1 is evaluated to v_1. If v_1 is `null`, the value of a is `null` with provenance e_1. Otherwise, e_2 is evaluated to v_2. If v_2 is `null`, the value of a is `null` with provenance e_2. Otherwise, the value of a is the bitwise AND of v_1 and v_2.

Let T_1, T_2 be the types of e_1, e_2 respectively.

Let $T = LCS(T_1, T_2)$. `&` has type $T, T \to T$. It is a type warning if either T_1 or T_2 is not a non-nullable numeric type.

The actual warning given will vary depending on whether the type of $e_i, i \in 1..2$ is a nullable numeric type or a non-numeric type. In the former case, we warn that we might encounter a `null` and propagate it; in the latter, we warn that this is truly a type mismatch. Same for other bitwise operators below, and similarly for other parts of the specification We may want to make this behavior normative.

⟨***BitwiseAndExpression***⟩ → ⟨*RelExpression*⟩
| ⟨*BitwiseAndExpression*⟩ `&` ⟨*RelExpression*⟩

10.16 Bitwise Xor

A *bitwise xor expression* is either a bitwise and [10.15] or the application of the bitwise xor operator, `^`, to two bitwise xor expressions.

Let $x \triangleq e_1$ `^` e_2.

Evaluation of x proceeds as follows:

The expression e_1 is evaluated to v_1. If v_1 is `null`, the value of a is `null` with provenance e_1. Otherwise, e_2 is evaluated to v_2. If v_2 is `null`, the value of x is `null` with provenance e_2. Otherwise, the value of x is the bitwise XOR of v_1 and v_2.

Let T_1, T_2 be the types of e_1, e_2 respectively.

Let $T = LCS(T_1, T_2)$. `^` has type $T, T \to T$. It is a type warning if either T_1 or T_2 is not a non-nullable numeric type.

⟨***BitwiseXorExpression***⟩ → ⟨*BitwiseAndExpression*⟩
| ⟨*BitwiseXorExpression*⟩ `^` ⟨*BitwiseAndExpression*⟩

10.17 Bitwise Or

A *bitwise or expression* is either a bitwise xor [10.16] or the application of the bitwise or operator, `|`, to two bitwise or expressions.

Let $o \triangleq e_1$ `|` e_2.

Evaluation of o proceeds as follows:

The expression e_1 is evaluated to v_1. If v_1 is `null`, the value of o is `null` with provenance e_1. Otherwise, e_2 is evaluated to v_2. If v_2 is `null`, the value of o is `null` with provenance e_2. Otherwise, the value of o is the bitwise OR of v_1 and v_2.

Let T_1, T_2 be the types of e_1, e_2 respectively.

Let $T = LCS(T_1, T_2)$. `|` has type $T, T \to T$. It is a type warning if either T_1 or T_2 is not a non-nullable numeric type.

⟨***BitwiseOrExpression***⟩ → ⟨*BitwiseXorExpression*⟩
| ⟨*BitwiseOrExpression*⟩ | ⟨*BitwiseXorExpression*⟩

10.18 Logical And

A *logical and expression* is either a bitwise or [10.17] or the application of the logical and operator, `and`, to two logical and expressions.

⟨***AndExpression***⟩ → ⟨*RelExpression*⟩
| ⟨*AndExpression*⟩ `and` ⟨*RelExpression*⟩

Let $a \triangleq e_1$ `and` e_2.

Evaluation of a proceeds as follows:

The expression e_1 is evaluated to v_1. If v_1 is `null`, the value of a is `null` with provenance e_1. If v_1 is false, the value of a is false. Otherwise, e_2 is evaluated to v_2. If v_2 is `null`, the value of a is `null` with provenance e_2. Otherwise, if the value of a is the result of v_2.

The type of a is `bool` [11.5.2].

Let T_1, T_2 be the types of $e1, e_2$ respectively. It is a type warning if either T_1 or T_2 is not of type `bool`.

The actual warning given will vary depending on whether the type of $e_i, i \in 1..2$ is `bool ?` or some other type $T \neq$ `bool`. In the former case, we warn that we might encounter a `null` and propagate it; in the latter, we warn that this truly type mismatch. Same for `or` below. We may want to make this behavior normative.

Note that logical and is not commutative.

10.19 Logical Or

A *logical or expression* is either a logical and expression [10.18] or the application of the logical or operator, `or`, to two logical or expressions.

⟨***OrExpression***⟩ → ⟨*AndExpression*⟩
| ⟨*OrExpression*⟩ `or` ⟨*AndExpression*⟩

Let $o \triangleq e_1$ `or` e_2.

Evaluation of o proceeds as follows:

The expression e_1 is evaluated to v_1. If v_1 is `null`, the value of o is `null` with provenance e_1. If v_1 is true, the value of o is true. Otherwise, e_2 is evaluated to v_2. If v_2 is `null`, the value of o is `null` with provenance e_2. Otherwise, if the value of o is the result of v_2.

The type of o is `bool` [11.5.2].

Let T_1, T_2 be the types of $e1, e_2$ respectively. It is a type warning if either T_1 or T_2 is not of type `bool`.

Note that logical or is not commutative.

10.20 Calls

A *call* invokes a function [5] with *arguments*.

⟨***FunctionCall***⟩ → ⟨*Ident*⟩ ⟨*ArgumentList*⟩

⟨***ArgumentList***⟩ → `(` (⟨*Expression*⟩(`,` ⟨*Expression*⟩)* `,`?)? `)`

Let $c \triangleq F\texttt{(}a_1\texttt{,}\ldots\texttt{,}a_k\texttt{)}$. Evaluation of c proceeds as follows.

Let f be the function denoted by F, and let $p_i, i \in 1\ldots k$ be its formal parameters, and let b_f be its body.

The arguments a_i are evaluated, in the order they appear in the source code to values $v_i, i \in 1..k$.

If any argument evaluates to `null`, c evaluates to `null` with provenance c.

Otherwise, $[p_i \leftarrow v_i]b_f$ is evaluated to r. The value of c is r.

It is a compile-time error if F is not bound to a function.

It is a compile-time error if f does not have exactly k formal parameters. Let T_{a_i} be the type of $a_i, i \in 1..k$, and let S_i be the type of the ith parameter of f. It is a type warning if T_{a_i} is not a subtype of S_i, $i \in 1..n$.

The type of c is the return type of f.

Note that even if no return type was explicitly written, there is an implicit return type, which is the type of the body of f, per section 5.

11 Types

11.1 The Error Type

The error type, written *, represents type errors. For all types $T, * < T$ and $T < *$.

*The alert reader will note that these rules destroy the type lattice. This is deliberate. The error type indicates a logical contradiction in the type structure of the program. No type correct program can ever contain any expression of type *.*

*The type * cannot be written explicitly in OPL. However, it is assigned by the type system whenever an expression does not have a valid type. For example, if a variable or formal parameter does not have a declared type, or if an expression does not respect the type rules requires for correctness.*

This allows us to give warnings when we cannot find a correct typing, but prevent the propagation of errors throughout the program.

11.2 Nullable Types

Null values are those values that are members of `Null` [11.3]. A type is *nullable* iff it contains null values. A *non-nullable* type is a type that does not contain any null value. We write T`?` to denote the nullable superset of type T.

We write $T!$ to denote the type expression that is constructed by removing all trailing `?` type constructors from T.

Note that $T!$ is a meta-notation used in this specification, and not part of the OPL language syntax.

We will sometimes speak of an expression being nullable (respectively non-nullable) if the type of the expression is nullable (respectively non-nullable).

The following equivalence always holds $X\texttt{??} = X\texttt{?}$.

The `?` operator is idempotent.

With non-nullable types, there are two possible approaches. In one, types are nullable by default; `null` is by default a member of any ordinary type T, and we designate a non-nullable type explicitly, with a notation like T`!`. The alternative is that types are non-nullable by default. We mark nullable types explicitly via the notation T`?`. The latter is safer, because it means that when you write a type, you must explicitly acknowledge that it might be `null`, or else get a warning. On the other hand, it is also more burdensome. In a language like OPL, where `null` is very often propagated across pipelines, there may be many warnings, and the need to be explicit might be seen as too painful. For now, we have opted for the safer approach, but this is a key design decision that is open for discussion.

Furthermore, these rules have implications for performance. If we can identify types as non-nullable, we can elide null checks.

11.3 The Null Type

The type `Null` is the set of pairs $(\texttt{null}, p)$ where `null` is a unique value and p is a *provenance*. A provenance is an expression [10], possibly combined with additional, implementation-specific information such as a trace and/or a runtime scope [3]. The expression will typically be the expression that gave rise to the `null` value in question.

11.4 Stream Signal Types

Signal types are the types of the elements of streams, and processed by builtin stream operators/combinators. Signal types have both mandatory well typed attributes, and dynamic attributes. The signal types form a subtype hierarchy; we write $T < S$ when signal type T is a subtype of signal type S.

The set of signal types is:

$\{Signal, Log, Metric, Span, Counter, UpdownCounter, Gauge, Histogram, ExponentialHistogram, Summary\}$.

For all signal types T, $T < Signal$.

The top of the signal type hierarchy is type $Signal$.

For all $T \in \{Counter, UpdownCounter, Gauge, Histogram, ExponentialHistogram, Summary\}$, $T < Metric$.

11.4.1 Signal

Type *Signal* includes the record valued fields resource and scope.

Are these two mandatory or optional? That is, since all their fields are optional, if they are empty need they be present all?

resource The resource record may contain the following fields, all of which are optional: $\{attributes : map[\texttt{string}, any], dropped\_attributes\_count : uint32, enitity\_ref : EntityRef[], schema\_url : \texttt{string}\}$, where the type *EntityRef* has the form $\{schema\_url : \texttt{string}, type : \texttt{string}, id\_keys : \texttt{string}[], description\_keys : \texttt{string}[]\}$ where the *type* and *id_keys* fields are mandatory, and the remaining fields are optional.

scope The scope record may contain the following fields, all of which are optional: $\{name : \texttt{string}, version : \texttt{string}, attributes : map, dropped\_attributes\_count : uint32, schema\_url : \texttt{string}\}$.

11.4.2 Log

*Log*s inherit all the fields of *Signal*. In addition a Log has a mandatory filed *time_unix_nano*, which is a an *int*64 [11.5.4] value indicating the time stamp of the log entry in nanoseconds *since when?*. If a *Log* value denotes an event, it must also have an *event_name* field of type `string`.

The following additional fields are optional: $\{body : any, attributes : map, trace\_id : byte[], trace\_id\_string : \texttt{string}, span_id : byte[], span\_id\_string : \texttt{string}, observed\_time\_unix\_nano : int64, severity\_number : int32, severity\_text : \texttt{string}, dropped\_attributes\_count : uint32, flags : uint32\}$.

The meanings of these should be defined somewhere.

11.4.3 Metric

*Metric*s inherit all the fields of *Signal*. In addition a *Span* has the following mandatory fields: $\{name : \texttt{string}, type : byte\}$.

A *Metric* may also include the following optional fields $\{description : \texttt{string}, unit : \texttt{string}, metadata : map\}$.

Types *Counter* and *upDownCounter* inherit all the fields of *Metric*. In addition a they may have the optional field $agregation\_temporality : int32$.

11.4.4 Span

*Span*s inherit all the fields of *Signal*. In addition a *Span* has the following mandatory fields: $\{name : \texttt{string}, trace\_id : byte[], span_id : byte[], start\_time\_unix\_nano : int64, end\_time\_unix\_nano : int64\}$.

A *Span* may also include the following optional fields $\{attributes : map, parent\_span\_id : byte[], trace\_state : \texttt{string}, status\_code : int32, status\_message : \texttt{string}, kind : int32, kind\_string : \texttt{string}, kind\_deprecated\_string : \texttt{string}, events : SpanEvent[], links :$

$SpanLink[], dropped\_attributes\_count : uint32, dropped\_events\_count : uint32, dropped\_links\_count : uint32, flags : uint32\}$.

The type *SpanEvent* consists of the required fields $\{span : Span, name :$ `string`$\}$. The *span* field is a back link to its parent *Span*. A *SpanEvent* may also include the optional fields $\{time\_unix\_nano : int64, attributes : map, dropped\_attributes\_count : int64\}$.

The type *SpanLink* consists of the required field $span : Span$ which is a back link to its parent *Span*, and may include the optional fields $\{trace\_id : byte[], span_id : byte[], trace\_state :$ `string`$, attributes : map, dropped\_attributes\_count : uint32, flags : uint32\}$.

11.5 Basic Types

Basic types are the types usable in expressions. They comprise the attribute types [11.5.1] and the types *any*, *byte*, *int*32, *uint*32 and *uint*64.

For any basic type $T, T < any$.

Type any is the supertype of all basic types.

11.5.1 Attribute Types

Attribute types are those types whose values may be assigned to attributes of signal types. Attribute are types transmitable via the protobuf protocol. These are the attribute types: $\{attributeValue,$ `string`$,$ `int64`$,$ `float64`$,$ `bool`$,$ `byte`$[], []attributeValue, map\}$.

The type *attributeValue* corresponds to the protobuf type *AnyValue*. Therefore, for all attribute types $T, T < attributeValue$.

In other words, attributeValue is a supertype of all attribute types.

The *map* type maps *string* values to values of other attribute types.

that is, it is parametric type, $map < string, attributeValue >$.

The remaining types all correspond to the types of the same name in the protobuf specification.

Optional type system insists on explicit cast from dynamic to any specific type. Later, we can introduce implicit type test (but not conversion) and provide distinct optional type system that allows dynamic to passed freely, relying on said implicit testing.

The separation is intentional, to optimize the data pipeline for OTEL signals, and to allow richer types in functions while using OTEL protobuf infrastructure.

11.5.2 bool

The predefined type `bool` contains the values true and false. A boolean value is equal only to itself. The value true is greater than false. The type `bool` is equal only to itself.

11.5.3 float64

The predefined type `float64` is consists of the set of IEEE double precision floating point values.

Equality and ordering are defined by the IEEE floating point standard.

11.5.4 Integer Types

The *integer types* are *signed integer types* `int32` and `int64`, and the *unsigned integer types* `uint32` and `uint64`.

Equality and ordering are defined by the laws of mathematics. However, arithmetic is not, because computations may overflow or underflow. Operations on integers do not indicate overflow or underflow.

Note on overflow: At the moment, no construct for dealing with this is defined in error policies [6.1], but we expect to choose trap/wrap behavior based on the error policy. Need to ensure that is compositional. We avoid bigInts as consumers cannot easily deal with them (no protobuf support except byte arrays).

The integer types are these:

int64 The predefined type `int64` consists of the integer values between -9,223,372,036,854,775,808 and 9,223,372,036,854,775,807, inclusive.

int32 The predefined type `int32` consists of the integer values between -2,147,483,648 and 2,147,483,647, inclusive.

uint64 The predefined type `uint64` consists of the integer values between 0 and 18,446,744,073,709,551,615, inclusive.

uint32 The predefined type `uint32` consists of the integer values between 0 and 4,294,967,295, inclusive.

11.5.5 string

The predefined type `string` consists of all sequences of UTF8 encoded characters.

11.5.6 regexp

The type `regex` consists of all valid IETF RFC 9485 regular expressions.

11.5.7 Date/time types

The type date/time types represent dates, times, durations or intervals as defined by ISO 8601.

`date` The predefined type `date` represents a date as defined by ISO 8601.

`time` The predefined type `time` represents a time as defined by ISO 8601.

`timestamp` The predefined type `timestamp` represents a combined date and time as defined by ISO 8601.

`duration` The predefined type `duration` represents a duration as defined by ISO 8601.

`interval` The predefined type `interval` represents an interval as defined by ISO 8601.

11.6 Function Types

A *function type* describes the type of a function [5].

A function type consists of a domain and a range. A *domain* is a product of a list of basic types [11.5], composed from the types of the formal parameters of a function, in the order they appear in the function declaration. A *range* is a result type of a function. We write a function type as $T_1, \ldots, T_n \to T_r$, where $T_1, \ldots, T_n$ is the domain and T_r is the range.

Function types are used internally in this specification, but have no direct syntactic representation.

Naturally, as OPL does not support higher order functions.

Function types are derived from function signatures.

12 Syntax

12.1 Lexical Rules

⟨***LINETERMINAL***⟩ ::= the ASCII LF character, also known as "newline"
| the ASCII CR character, also known as "return"
| the ASCII CR character followed by the ASCII LF character

⟨***SLCOMMENT***⟩ ::= `//` any character until the first encountering of ⟨*LINETERMINAL*⟩ or ⟨*EOF*⟩

⟨***MLCOMMENT***⟩ ::= `/*` any character until the first encountering of `*/`

⟨***WS***⟩ ::= the ASCII SP character, also known as "space"
| the ASCII HT character, also known as "horizontal tab"
| the ASCII FF character, also known as "form feed"
| ⟨*LINETERMINAL*⟩
| ⟨*SLCOMMENT*⟩
| ⟨*MLCOMMENT*⟩

⟨***IDENTSTART***⟩ ::= `A` ... `Z`
| `a` ... `z`
| `_`

⟨***IDENTPART***⟩ ::= `A` ... `Z`
| `a` ... `z`
| `0` ... `9`
| `_`

⟨***IDENT***⟩ ::= ⟨*IDENTSTART*⟩ ⟨*IDENTPART*⟩*

⟨***TYPEPARAM***⟩ ::= `$`⟨*IDENT*⟩

⟨***DECIMAL***⟩ ::= `0` ... `9`

⟨***HEX***⟩ ::= `0` ... `9`
| `a` ... `f`
| `A` ... `F`

⟨***FRAC***⟩ ::= ⟨*DECIMAL*⟩+

⟨***EXPO***⟩ ::= (`e` | `E`) (`+` | `-`)? ⟨*FRAC*⟩

⟨***FLOAT***⟩ ::= ⟨*FRAC*⟩ `.` ⟨*FRAC*⟩? ⟨*EXPO*⟩?
| ⟨*FRAC*⟩ ⟨*EXPO*⟩?
| `.` ⟨*FRAC*⟩ ⟨*EXPO*⟩?

⟨***INT***⟩ ::= (`1` ... `9`)⟨*FRAC*⟩?
| `0` lookahead ∉ `0` ... `9`
| `0` (`x` | `X`) ⟨*HEX*⟩+

⟨***ESCAPE***⟩ ::= the ASCII SP character, also known as “space”
| one of `t`, `v`, `r`, `n`, `f`, `b`, `\`, `'`, `"`

⟨***DQCHAR***⟩ ::= any one character except `"`, ⟨*LINETERMINAL*⟩ or `\`
| `\` ⟨*ESCAPE*⟩

⟨***DQSTRING***⟩ ::= `"` ⟨*DQCHAR*⟩* `"`

⟨***SQSTRING***⟩ ::= `'` ⟨*SQCHAR*⟩* `'`

⟨***SQBYTE***⟩ ::= any one character in the range 0-255 except `'`, ⟨*LINETERMINAL*⟩ or `\`
| `\` ⟨*ESCAPE*⟩

⟨***SQCHAR***⟩ ::= any one character except `'`, ⟨*LINETERMINAL*⟩ or `\`
| `\` ⟨*ESCAPE*⟩

12.2 Grammar

⟨***OPLProgram***⟩ → ⟨*LevelDecl*⟩? (⟨*Decl*⟩
| ⟨*Policy*⟩)* ⟨*Pipeline*⟩+

⟨***LevelDecl***⟩ → `requires level` >= ⟨*NumberLiteral*⟩

⟨***Decl***⟩ → ⟨*FunctionDeclaration*⟩
| ⟨*LetBinding*⟩

⟨***FunctionDeclaration***⟩ → `func` ⟨*FunctionSignature*⟩ `=` ⟨*Expression*⟩

⟨***FunctionSignature***⟩ → ⟨*IDENT*⟩ ⟨*ParameterList*⟩ ⟨*TypeSigSuffix*⟩?

⟨***LetBinding***⟩ → `let` ⟨*Ident*⟩ ⟨*TypeSuffix*⟩? = (`receiver` ⟨*IDENT*⟩ ⟨*Block*⟩
| `exporter` ⟨*IDENT*⟩ ⟨*Block*⟩)

⟨***Policy***⟩ → ⟨*ErrorPolicy*⟩
| ⟨*BatchPolicy*⟩
| ⟨*RetryPolicy*⟩
| ⟨*RateLimitPolicy*⟩
| ⟨*TimeoutPolicy*⟩

⟨***ErrorPolicy***⟩ → `policy error` ⟨*ErrorStrategy* ⟩

⟨***ErrorStrategy***⟩ → `strategy` ⟨*ErrorAction*⟩

⟨***ErrorAction***⟩ → `drop_signal`
| `route_to` ⟨*Destination*⟩
| `propagate`
| `log_and_continue`

⟨***BatchPolicy***⟩ → `policy batch` ⟨*BatchConfig* ⟩

⟨***BatchConfig***⟩ → `{ size :` Integer `, timeout :` ⟨*Duration*⟩ `, flush_on :`
⟨*FlushTrigger*⟩ `}`

⟨***FlushTrigger***⟩ → `size`
| `timeout`
| `any`

⟨***Duration***⟩ → ⟨*TaggedLiteral*⟩

⟨***TimeUnit***⟩ → `ms`
| `s`
| `m`
| `h`

⟨***RetryPolicy***⟩ → `policy retry` ⟨*RetryConfig* ⟩

⟨***RetryConfig***⟩ → `{ max_attempts :` Integer `, backoff :` ⟨*BackoffStrategy*⟩
`, retry_on : [` ⟨*ErrorCondition*⟩ (`,` ⟨*ErrorCondition*⟩)*
`](, timeout :` ⟨*Duration*⟩ `}`

⟨***BackoffStrategy***⟩ → `fixed` ⟨*Duration*⟩ `:` ⟨*Duration*⟩
| `exponential` ⟨*ExponentialConfig*⟩
| `jitter` ⟨*JitterConfig*⟩

⟨***ExponentialConfig***⟩ → `initial :` ⟨*Duration*⟩ `, max :` ⟨*Duration*⟩ `, multiplier`
`:` ⟨*FLOAT*⟩

⟨***JitterConfig***⟩ → `base :` ⟨*BackoffStrategy*⟩ `, jitter_factor :` ⟨*FLOAT*⟩

⟨***ErrorCondition***⟩ → `network`
| `timeout`
| `rate_limit`
| `server_error`
| `unavailable`
| `*`

⟨***RateLimitPolicy***⟩ → `policy rate_limit` ⟨*RateLimitConfig* ⟩

⟨***RateLimitConfig***⟩ → `{ strategy :` ⟨*RateLimitStrategy*⟩ `, on_limit :` ⟨*LimitAction*⟩
`}`

⟨***RateLimitStrategy***⟩ → `token_bucket` ⟨*TokenBucketConfig*⟩
| `fixed_window` ⟨*FixedWindowConfig*⟩
| `sliding_window` ⟨*SlidingWindowConfig*⟩

⟨***TokenBucketConfig***⟩ → `rate :` ⟨*Rate*⟩ `, burst :` Integer

⟨***FixedWindowConfig***⟩ → `rate :` ⟨*Rate*⟩ `, window :` ⟨*Duration*⟩

⟨***SlidingWindowConfig***⟩ → `rate :` ⟨*Rate*⟩ `, window :` ⟨*Duration*⟩ `, granularity :` ⟨*Duration*⟩

⟨***Rate***⟩ → Integer "/" ⟨*TimeUnit*⟩

⟨***LimitAction***⟩ → `drop`
| `route_to` Destination
| `block`
| `back_pressure`

⟨***TimeoutPolicy***⟩ → `policy timeout` ⟨*TimeoutConfig* ⟩

⟨***TimeoutConfig***⟩ → `{ operation :` ⟨*Duration*⟩ `, pipeline :` ⟨*Duration*⟩ `, on_timeout :` ⟨*TimeoutAction*⟩ `}`

⟨***TimeoutAction***⟩ → `fail`
| `route_to` ⟨*Destination*⟩

⟨***Block***⟩ → `{` ⟨*PropertyList*⟩ `}`

⟨***Pipeline***⟩ → ⟨*Source*⟩ (| ⟨*Stage*⟩)* ⟨*Terminal*⟩?

⟨***Source***⟩ → ⟨*Ident*⟩

⟨***Stage***⟩ → ⟨*OperatorCall*⟩
| ⟨*Control*⟩
| ⟨*PolicyStage*⟩
| ⟨*StreamComb*⟩

⟨***OperatorCall***⟩ → ⟨*Ident*⟩ ⟨*Args*⟩?

⟨***Control***⟩ → ⟨*IfExpression*⟩
| ⟨*Union*⟩
| ⟨*ForkExpression*⟩

⟨***StreamComb***⟩ → ⟨*Union*⟩
| ⟨*Cast*⟩

⟨***Cast***⟩ → `accept` ⟨*TypeList*⟩
| `reject` ⟨*TypeList*⟩

⟨***Union***⟩ → `union` ⟨*Pipeline*⟩*

⟨***Terminal***⟩ → `route_to`⟨*Ident*⟩
| `drop`

⟨***ParameterList***⟩ → `(` ⟨*Parameter*⟩(`,` ⟨*Parameter*⟩)* `,`? `)`

⟨***Parameter***⟩ → ⟨*IDENT*⟩ ⟨*TypeSigSuffix*⟩?

⟨***PropertyList***⟩ → (Property (`,` Property)*)?

⟨***PropertyName***⟩ → ⟨*IDENT*⟩
| ⟨*StringLiteral*⟩

⟨***TypeSigSuffix***⟩ → `:` ⟨*TypeSig*⟩

⟨***NonNullableTypeSig***⟩ → ⟨*TYPEPARAM*⟩
| ⟨*RefType*⟩
| ⟨*BuiltinType*⟩
| ⟨*StructType*⟩
| ⟨*ListType*⟩
| ⟨*MapType*⟩

⟨***TypeSig***⟩ → ⟨*NonNullableTypeSig* ⟩ `?`

⟨***BuiltinType***⟩ → `bool`
| `number`
| `string`
| `byte`
| `uint32`
| `uint64`
| `int32`
| `int64`
| `float64`
| `any`
| `attributeValue`
| `map`

⟨***PropertySig***⟩ → ⟨*PropertyName*⟩ `:` ⟨*TypeSig*⟩

⟨***RefType***⟩ → ⟨*Qualifier*⟩ ⟨*IDENT*⟩ (⟨*TypeArgumentList*⟩)?

⟨***Expression***⟩ → ⟨*IfExpression*⟩
| ⟨*BitwiseOrExpression*⟩

⟨***IfExpression***⟩ → `if` ⟨*Expression*⟩ `(` ⟨*Pipeline*⟩ `)` (`else if (` ⟨*Pipeline*⟩ `)`) *
(`else (` ⟨*Pipeline*⟩ `)`)?

⟨***OrExpression***⟩ → ⟨*AndExpression*⟩
| ⟨*OrExpression*⟩ `or` ⟨*AndExpression*⟩

⟨***AndExpression***⟩ → ⟨*RelExpression*⟩
| ⟨*AndExpression*⟩ `and` ⟨*RelExpression*⟩

⟨***BitwiseOrExpression***⟩ → ⟨*BitwiseXorExpression*⟩
| ⟨*BitwiseOrExpression*⟩ `|` ⟨*BitwiseXorExpression*⟩

⟨***BitwiseXorExpression***⟩ → ⟨*BitwiseAndExpression*⟩
| ⟨*BitwiseXorExpression*⟩ `^` ⟨*BitwiseAndExpression*⟩

⟨***BitwiseAndExpression***⟩ → ⟨*RelExpression*⟩
| ⟨*BitwiseAndExpression*⟩ `&` ⟨*RelExpression*⟩

⟨***RelExpression***⟩ → ⟨*AdditiveExpression*⟩
| ⟨*RelExpression*⟩ ⟨*RelOp*⟩ ⟨*AdditiveExpression*⟩

⟨***RelOp***⟩ → `==`
| `!=`
| `<`
| `>`
| `<=`
| `>=`

⟨***AdditiveExpression***⟩ → ⟨*MultiplicativeExpression*⟩
| ⟨*AdditiveExpression*⟩ ⟨*AdditiveOp*⟩ ⟨*MultiplicativeExpression*⟩

⟨***AdditiveOp***⟩ → `+`
| `-`

⟨***MultiplicativeExpression***⟩ → ⟨*UnaryExpression*⟩
| ⟨*MultiplicativeExpression*⟩ ⟨*MultiplicativeOp*⟩ ⟨*UnaryExpression*⟩

⟨***MultiplicativeOp***⟩ → `*`
| `/`
| `%`

⟨***ForkExpression***⟩ → `fork (`⟨*Pipeline*⟩ `)` *

⟨***UnaryExpression***⟩ → ⟨*NumberLiteral*⟩
| `-` ⟨*UnaryExpression*⟩
| `~` ⟨*UnaryExpression*⟩
| `not` ⟨*UnaryExpression*⟩
| ⟨*MemberExpression*⟩

⟨***ParenthesizedExpression***⟩ → `(` ⟨*Expression*⟩
| ⟨*ExpressionTypeSigSuffix*⟩ `)`

⟨***MemberExpression***⟩ → ⟨*IndexExpression*⟩
| ⟨*AttributeSelectionExpression*⟩
| ⟨*FunctionCall*⟩
| ⟨*ExtensionExpression*⟩
| ⟨*PrimitiveExpression*⟩

⟨***IndexExpression***⟩ → ⟨*MemberExpression*⟩ `[` ⟨*Expression*⟩ `]`

⟨***AttributeSelectionExpression***⟩ → ⟨*MemberExpression*⟩ `.` ⟨*PropertyName*⟩

⟨***FunctionCall***⟩ → ⟨*Ident*⟩ ⟨*ArgumentList*⟩

⟨***ArgumentList***⟩ → `(` (⟨*Expression*⟩(`,` ⟨*Expression*⟩)* `,`?)? `)`

⟨***ExtensionExpression***⟩ → ⟨*MemberExpression*⟩ `.` ⟨*StructLiteral*⟩

⟨***PrimitiveExpression***⟩ → ⟨*IdentifierExpression*⟩
| ⟨*StringLiteral*⟩
| ⟨*TaggedLiteral*⟩
| `true`
| `false`
| `null`
| `(` ⟨*ParenthesizedExpression*⟩

⟨***IdentifierExpression***⟩ → ⟨*IDENT*⟩

⟨***Property***⟩ → ⟨*PropertyName*⟩ `:` ⟨*PropertyValue*⟩

⟨***PropertyValue***⟩ → ⟨*Scalar*⟩
| ⟨*Block*⟩

⟨***Scalar***⟩ → ⟨*NumberLiteral*⟩
| ⟨*StringLiteral*⟩
| `true`
| `false`
| `null`
| ⟨*ScalarArray*⟩

⟨***ScalarArray***⟩ → `[` ⟨*Scalar*⟩ (`,` ⟨*Scalar*⟩)* `]`

⟨***StringLiteral***⟩ → ⟨*DQSTRING*⟩
| ⟨*SQSTRING*⟩

⟨***NumberLiteral***⟩ → ⟨*FLOAT*⟩
| ⟨*INT*⟩

⟨***TaggedLiteral***⟩ → ⟨*IDENT*⟩ ⟨*StringLiteral*⟩
| ⟨*NumberLiteral*⟩ ⟨*IDENT*⟩

⟨***MapElement***⟩ → ⟨*Expression*⟩ `:` ⟨*Expression*⟩

13 Version History

- 0.01. Initial public version.